Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE
Slot: B

ASSIGNMENT 1 – CO1

Nature of Assignment: Individual **Duration: 1 Week** **Total Marks: 50**

Student Name: VISHAL.V **Reg. No: 192419033**

CO1: Foundation and history of AI, intelligent agents (agents and environments, rationality, nature of environments, structure of agents), problem-solving agents, problem formulation, searching for solutions, uninformed search strategies and search with partial information.

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

Scenario — Hospital Medicine-Delivery Robot

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The building is modelled as a graph of locations (junctions, corridors, lifts, ward doors). The robot may move between connected locations and every move has a travel-time cost.

As new wings are added, the state space of the robot may grow:

1,000 → 10,000 → 100,000 → 1,000,000+ states

You are required to investigate, design and compare uninformed search strategies by which the robot finds a route from the pharmacy to a requested ward as the hospital grows.

Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building adjacency lists or a duplicate-detection structure) must be included in your efficiency analysis.

Design Challenge (Day 4): The robot's localisation sensor now fails intermittently, so the robot no longer knows exactly where it is. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

Before designing algorithms, define the agent and the problem:

- Specify the PEAS description of the robot and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent), justifying each choice
- State which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the robot follows and why it is a rational problem-solving agent. Relate the problem type briefly to the early history of AI problem solving
- Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** uninformed search strategies for finding a route. Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.

Your proposed approaches must include:

- At least one iterative approach (queue- or stack-based)
- At least one recursive approach
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required, such as adjacency-list construction or an explored set

You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies are part of the assessment.

Evidence Expected: PEAS and environment classification, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

TASK 2 – Identify the Basic Operation (2 Marks)

For each proposed algorithm, identify the basic operation that predominantly determines its execution time.

Discuss:

- What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)
- Where it occurs in the algorithm

- Why it is appropriate for efficiency analysis

- How the number of times it executes may change as the state-space size increases

Do not simply name an operation. Relate it to the behaviour of the algorithm.

Evidence Expected: Correct identification and justification of the basic operation for each algorithm.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

Derive the time and space complexity for each proposed approach using mathematical analysis.

Discuss:

- The input size (branching factor b, shallowest-goal depth d, maximum depth m, optimal cost C^*, minimum step cost ε)
- The number of times the basic operation executes, derived by summing the nodes at each level
- The mathematical expression for time and for memory (frontier size)
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.

Evidence Expected: A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

Analyse each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the best case (for example, the goal lies on the first branch explored)
- What condition produces the average case
- What condition produces the worst case (for example, goal at maximum depth, goal unreachable, cheap-edge detours)

- How the number of nodes generated changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.

Evidence Expected: A justified best-, average- and worst-case analysis supported by mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ.

Discuss:

- How each algorithm behaves as the state space becomes very large
- How the growth rates of the approaches differ in time and in memory
- What the growth means when the hospital grows from 1,000 to 1,000,000+ states, for example, whether memory or time becomes the limiting factor first

Interpret the behaviour in the context of the hospital robot rather than merely stating the notation.

Evidence Expected: Correct asymptotic representation and a meaningful comparison of growth behaviour.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm.

Identify clearly:

- The size of the original problem (depth limit or remaining depth)
- The size of the recursive subproblem
- The number of recursive calls (related to the branching factor)
- The work performed outside the recursive calls
- The base case
- How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook.

Evidence Expected: A correct recurrence with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the recursive algorithm as depth or branching factor increases
- How the result extends to iterative deepening, where the recursive search is repeated for increasing limits

Show the analytical process, not only the final result.

Evidence Expected: A complete solution with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms (4 Marks)

Implement all proposed algorithms in Python and test them experimentally.

Use generated hospital graphs of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 states

Ensure that:

- All algorithms return correct routes
- The same or equivalent graphs and start/goal pairs are used for comparison
- Testing conditions are consistent and the random seeds are reported
- Test cases include a reachable goal, an unreachable goal and a start equal to the goal
- The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.

Evidence Expected: Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

TASK 9 – Record Experimental Performance (3 Marks)

Collect and present performance data for each algorithm across the different state-space sizes.

Record, with code and outputs as proof:

- Nodes generated and nodes expanded
- Maximum frontier size (memory proxy)
- Solution depth and path cost, and whether the path is optimal
- Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.

Evidence Expected: Systematically collected experimental data with clear tables and graphs.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

Compare your theoretical analysis with the experimental results.

Discuss:

- Whether the observed growth of nodes generated and of frontier size agrees with the theoretical complexity
- Which algorithm scales better as the state space increases
- Any differences between theory and observation, and the possible reasons (graph structure, duplicates, tie-breaking, language overhead)

Interpret the results rather than simply reproducing tables or graphs.

Evidence Expected: A reasoned comparison between theoretical and observed performance.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for the hospital robot.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives

- Its theoretical efficiency and experimentally observed performance
- Any preprocessing cost, trade-offs or limitations
- Whether it remains practical as the hospital grows

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected: A technically justified recommendation supported by analysis and experiment.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

After a power restart or sensor fault, the robot no longer knows exactly where it is. It knows only that it is at one of *k* possible junctions. It must still reach the pharmacy docking station, whichever junction it is actually at.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

- What has changed from the original problem and which original assumptions (fully observable, known start state) are affected?
- Why does the robot's knowledge now form part of the state?

B. Evaluation of the Original Design – 2 Marks

- Can your original algorithm still be applied? What does a "state" become?
- How large can the new search space be for *k* possible locations, and is the original design acceptable?

C. Adaptation or Redesign – 2 Marks

- Describe the revised strategy (for example, search over belief states, or search with local sensing and a conditional plan), what has changed and the effect on complexity
- If you retain the original solution, give sufficient technical justification

D. Evidence-Based Defence – 2 Marks

- Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for k = 2, 4, 8, with and without a local sensor), with code and output as proof.

MARK DISTRIBUTION – ASSIGNMENT_1

Task	Marks
Task 1 – Agent and Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

SOLUTION: -

TASK 1 – Design the Agent and Search Algorithms

1.1 PEAS Description

PEAS stands for Performance, Environment, Actuators and Sensors. It describes how an intelligent agent works.

Component	Description
Performance	Deliver medicines safely, quickly and with minimum travel cost
Environment	Hospital rooms, corridors, lifts, junctions and wards
Actuators	Wheels, motors, brakes and navigation system
Sensors	Camera, location sensor, obstacle detector and distance sensor

1.2 Environment Classification

The hospital environment can be classified as follows:

Type	Classification	Reason
Observability	Partially observable	The robot may not know every location and obstacle
Determinism	Mostly deterministic	Movement is predictable, but unexpected obstacles may occur
Episodic/Sequential	Sequential	Each movement affects the next decision
Static/Dynamic	Dynamic	People and obstacles may change the routes
Discrete/Continuous	Discrete	Hospital locations are represented as separate nodes
Agents	Single-agent	One robot performs the medicine delivery task

1.3 AI Approach

The robot follows the acting rationally approach.

It observes the hospital, compares available routes and selects a suitable path to deliver medicines safely with minimum travel cost.

This is related to the early development of AI, where problem-solving methods and search algorithms were developed to solve problems systematically.

1.4 Problem Formulation

Component	Description
States	Different hospital locations
Initial state	Central pharmacy
Actions	Move between connected locations
Transition model	Describes the next location after movement
Goal test	Check whether the requested ward is reached
Path cost	Total travel cost of the selected route

1.5 Proposed Search Algorithms

Algorithm 1: Breadth-First Search (BFS)

Basic idea: BFS explores all nearby locations first before moving to locations further away.

It uses a FIFO queue (First In, First Out).

Pseudocode:

BFS(Graph, Start, Goal)

Create an empty queue

Create a visited set

Add Start to queue

Mark Start as visited

While queue is not empty:

Remove first node from queue

If node is Goal:

Return path

For each neighbour:

If neighbour is not visited:

Mark as visited

Add neighbour to queue

Return failure

Why choose BFS?

BFS is useful for small hospital networks because it finds the path with the fewest movements when every connection has equal cost.

Algorithm 2: Depth-First Search (DFS)

Basic idea: DFS follows one path as far as possible before returning and exploring another path.

It can be implemented using recursion.

Pseudocode:

DFS (Node, Goal)

If Node is Goal:

Return success

Mark Node as visited

For each neighbour:

If neighbour is not visited:

If DFS (neighbour, Goal):

Return success

Return failure

Why choose DFS?

DFS uses less search memory than BFS in many search-tree situations. It is useful for exploring large networks, but it does not guarantee the shortest route.

Algorithm 3: Uniform Cost Search (UCS)

Basic idea: UCS always explores the location with the lowest total travel cost first.

It uses a priority queue.

Pseudocode:

```
UCS (Graph, Start, Goal)
Create a priority queue
Set cost of Start = 0
Insert Start into queue
While queue is not empty:
    Remove node with lowest cost
    If node is Goal:
        Return path and cost
    For each neighbour:
        Calculate new cost
        If new cost is smaller:
            Update the cost
            Add neighbour to queue
Return failure
```

Why choose UCS?

UCS is useful for hospital delivery because different corridors and routes may have different travel costs. It finds the minimum-cost route when costs are non-negative.

1.6 Preprocessing Requirements

Before running the algorithms, the hospital network must be represented as a graph.

- Each hospital location is represented as a node.
- Each connected route is represented as an edge.
- An adjacency list stores the connected locations.
- A visited set prevents repeated exploration.
- A priority queue is used for UCS.

For example:

```
graph = {
    'Pharmacy': ['A', 'B'],
    'A': ['Pharmacy', 'C'],
    'B': ['Pharmacy', 'D'],
    'C': ['A', 'Ward'],
    'D': ['B', 'Ward'],
    'Ward': ['C', 'D']
}
```

This makes it easier for the robot to identify connected locations and search for routes.

TASK 2 – Identify the Basic Operation

The basic operation is the main activity that determines how much time an algorithm takes.

Algorithm	Basic operation	Where it occurs
BFS	Node expansion	When a node is removed from the queue and its neighbours are checked
DFS	Recursive node expansion	When the algorithm visits a node and calls itself for unvisited neighbours
UCS	Priority queue operations	When the lowest-cost node is removed and neighbouring costs are updated

Explanation:

- In BFS, node expansion happens for every visited location.
- In DFS, recursive calls explore connected locations one by one.
- In UCS, priority queue operations determine which location is explored next.

As the hospital grows, more locations and connections must be processed. Therefore, the number of these operations increases.

TASK 3 – Derive Time and Space Complexity Mathematically

3.1 Important Terms

Before calculating complexity, understand these terms:

Symbol	Meaning
V	Total number of hospital locations (vertices)
E	Total number of connections (edges)
b	Number of possible next locations (branching factor)
d	Depth of the nearest goal
m	Maximum search depth
C*	Minimum total cost of the solution
ϵ	Minimum cost of one movement

3.2 BFS Complexity

BFS uses a queue to explore locations level by level.

At each level, it checks the connected locations.

For a search tree:

$$1 + b + b^2 + \cdots + b^d$$

Using the geometric series:

$$T(n) = 1 + b + b^2 + \cdots + b^d$$

$$T(n) = \frac{b^{d+1} - 1}{b - 1}$$

Therefore:

Time complexity:

$$O(b^d)$$

Space complexity:

$$O(b^d)$$

For our hospital graph, where duplicate detection is used, the tighter graph-based bounds are:

$$T(V,E) = O(V+E)$$

$$S(V,E) = O(V)$$

The adjacency list itself requires $O(V+E)$ preprocessing space.

3.3 DFS Complexity

DFS explores one path deeply before returning to another path.

For a search tree:

$$T(n) = 1 + b + b^2 + \cdots + b^m$$

Therefore:

$$T(n) = O(b^m)$$

Its search stack and visited information require:

$$S(n) = O(m)$$

For our hospital graph, recursive DFS with a visited set explores every vertex and edge at most a constant number of times.

$$T(V,E) = O(V+E)$$

$$S(V,E) = O(V)$$

The recursive call stack can use up to $O(V)$ memory.

3.4 UCS Complexity

UCS selects the node with the lowest total path cost using a priority queue.

The number of nodes explored depends on the path cost and the minimum step cost.

For a search tree, the approximate depth of the optimal solution is bounded by:

$$D = \left\lfloor \frac{C^*}{\epsilon} \right\rfloor$$

The number of generated nodes is bounded by:

$$1 + b + b^2 + \cdots + b^D$$

Thus:

$$T(n) = O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$$

A graph-based UCS implementation using a binary heap has:

$$T(V,E) = O\big((V+E)\log V\big)$$

$$S(V,E) = O(V)$$

3.5 Preprocessing Cost

Before searching, we construct an adjacency list.

- Reading all locations requires O(V).
- Reading all connections requires O(E).
- Creating the adjacency list requires O(V+E).
- Creating the visited set requires up to O(V) space.

Therefore:

$$T_{\text{preprocessing}} = O(V+E)$$

For all three algorithms, the adjacency list can be reused instead of being created again.

3.6 Overall Complexity Comparison

Algorithm	Time complexity	Search space
BFS	O(V+E)	O(V)
DFS	O(V+E)	O(V)
UCS	O((V+E) \log V)	O(V)

Note: These are graph-based complexities with adjacency lists and duplicate detection. They differ from the general search-tree bounds because a graph search avoids repeatedly expanding already visited locations.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour

Each algorithm performs differently depending on where the requested ward is located and how the hospital is connected.

4.1 BFS

- Best case: The goal is directly connected to the pharmacy. Only a few nodes are explored.
- Average case: The goal is somewhere in the middle of the network. Several nodes are explored.
- Worst case: The goal is far away or unreachable. Almost all reachable nodes and connections may be explored.

4.2 DFS

- Best case: The goal is found along the first path explored.
- Average case: The robot explores some paths before finding the goal.
- Worst case: The goal is in the last path explored or is unreachable. The algorithm may visit almost the entire network.

4.3 UCS

- Best case: The goal is reached through a low-cost route early in the search.
- Average case: Several locations are explored based on their accumulated travel costs.
- Worst case: The goal is unreachable or many cheaper paths must be explored before reaching it.

Comparison:

Algorithm	Best case	Worst case
BFS	Very few nodes explored	O(V+E)
DFS	Goal found in first path	O(V+E)
UCS	Goal reached with little exploration	O((V+E) \log V)

The actual performance also depends on graph structure, branching factor, goal position and duplicate detection.

TASK 5 – Perform Asymptotic Analysis

Asymptotic analysis explains how an algorithm behaves when the input size becomes very large.

The three common notations are:

- Big O: Upper bound of growth.
- Big Omega (Ω) : Lower bound of growth.
- Big Theta (Θ): Tight bound of growth.

Comparison of Algorithms:

Algorithm	Graph-based time	Search memory
BFS	O(V+E)	O(V)
DFS	O(V+E)	O(V)
UCS	O((V+E) \log V)	O(V)

Behaviour as the Hospital Grows:

The hospital may grow from 1,000 to more than 1,000,000 locations.

- BFS remains efficient for finding the shortest path in terms of number of connections, but its queue can become large.
- DFS uses a stack and explores fewer locations in some cases, but it may take a long time to find the correct route.
- UCS requires additional priority queue operations, but it is useful when routes have different travel costs.

Conclusion

For a large hospital network, memory and execution time become important. BFS and DFS have linear graph-search complexity, while UCS has additional priority queue overhead. However, UCS is more suitable when minimum travel cost is important.

TASK 6 – Formulate the Recurrence Relation

For this task, we use recursive DFS.

Recursive DFS Behaviour:

DFS starts at one location, visits its neighbours and continues recursively until it reaches the goal or has no unvisited neighbours.

Let:

- V = Number of remaining unvisited locations.
- E = Number of remaining connections.
- deg(v) = Number of neighbours of the current location.

The recurrence relation is:

$$T(V,E) = T\big(V-1, E-deg(v)\big) + O(deg(v)+1)$$

Explanation

1. The original problem contains V locations.
2. DFS visits one location.
3. It checks all the connected neighbours.
4. It recursively explores the remaining unvisited locations.
5. Each location is visited only once because of the visited set.

Base Case

When there are no more unvisited locations:

$$T(0,0) = O(1)$$

Final Recurrence

For a simple upper-bound representation, if each node has at most \(b\) neighbours:

$$T(V) = T(V-1) + O(b)$$

Expanding:

$$T(V) = T(V-1) + b$$

$$= T(V-2) + 2b$$

$$= T(V-3) + 3b$$

Continuing:

$$T(V) = T(0) + Vb$$

$$T(V) = O(Vb)$$

For a graph with bounded degree, \(b\) is constant, so:

$$\boxed{T(V) = O(V)}$$

More precisely, the adjacency-list graph traversal takes:

$$\boxed{T(V,E) = O(V+E)}$$

This shows that recursive DFS can efficiently explore a hospital graph when visited-node tracking is used.

TASK 7 – Solve the Recurrence Relation

We use the recurrence obtained in Task 6:

$$T(V) = T(V-1) + O(b)$$

For simplicity, assume the cost of checking each node is proportional to \(b\).

Step 1: Expand the recurrence

$$T(V) = T(V-1) + b$$

$$T(V-1) = T(V-2) + b$$

Substituting:

$$T(V) = T(V-2) + 2b$$

Again:

$$T(V) = T(V-3) + 3b$$

Step 2: General expression

After k expansions:

$$T(V) = T(V-k) + kb$$

When k=V:

$$T(V) = T(0) + Vb$$

Step 3: Apply the base case

Since:

$$T(0) = O(1)$$

We get:

$$T(V) = O(1) + Vb$$

Therefore:

$$\boxed{T(V) = O(Vb)}$$

For a hospital graph with a bounded number of neighbours per location, \(b\) is constant.

Hence:

$$\boxed{T(V) = O(V)}$$

For the complete adjacency-list graph traversal, the more accurate complexity is O(V+E).

Extension to Iterative Deepening

Iterative Deepening Search repeatedly runs depth-limited DFS with increasing limits.

Its total time is:

$$T(d) = \sum_{i=0}^{d} O\left(b^i\right)$$

$$T(d) = O(1 + b + b^2 + \cdots + b^d)$$

Therefore:

$$\boxed{T(d) = O(b^d)}$$

It uses less memory than BFS, but repeats some searches at each depth.

Conclusion

The recurrence shows how the number of recursive operations increases as the hospital network grows. Recursive DFS is efficient for graph traversal when duplicate locations are avoided.

TASK 8 – Implement and Test the Algorithms

We can implement all three algorithms using Python.

The following program includes:

- BFS
- Recursive DFS
- UCS
- Path calculation
- Cost calculation
- Node expansion count
- Execution time

Python Program

```
import heapq
import time
from collections import deque
# Hospital network
graph = {
    'P': [('A', 2), ('B', 4)],
    'A': [('P', 2), ('C', 3), ('D', 5)],
    'B': [('P', 4), ('D', 2)],
    'C': [('A', 3), ('W', 4)],
    'D': [('A', 5), ('B', 2), ('W', 2)],
    'W': [('C', 4), ('D', 2)]
}
# Breadth First Search
def bfs(start, goal):
    queue = deque([(start, [start])])
    visited = {start}
    expanded = 0
```

```
    while queue:
        node, path = queue.popleft()
        expanded += 1
        if node == goal:
            return path, expanded
        for neighbour, cost in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append((neighbour, path + [neighbour]))
    return None, expanded
# Recursive Depth First Search
def dfs(start, goal):
    visited = set()
    expanded = 0
    def search(node, path):
        nonlocal expanded
        visited.add(node)
        expanded += 1
        if node == goal:
            return path
        for neighbour, cost in graph[node]:
            if neighbour not in visited:
                result = search(neighbour, path + [neighbour])
                if result:
                    return result
        return None
    return search(start, [start]), expanded
# Uniform Cost Search
def ucs(start, goal):
```

```
    queue = [(0, start, [start])]
    best = {start: 0}
    expanded = 0
    while queue:
        cost, node, path = heapq.heappop(queue)
        if cost != best[node]:
            continue
        expanded += 1
        if node == goal:
            return path, cost, expanded
        for neighbour, weight in graph[node]:
            new_cost = cost + weight
            if new_cost < best.get(neighbour, float('inf')):
                best[neighbour] = new_cost
                heapq.heappush(
                    queue,
                    (new_cost, neighbour, path + [neighbour])
                )
    return None, float('inf'), expanded
# Test all algorithms
start = 'P'
goal = 'W'
for name, algorithm in [
    ("BFS", bfs),
    ("DFS", dfs)
]:
    begin = time.perf_counter()
    path, nodes = algorithm(start, goal)
    elapsed = time.perf_counter() - begin
```

```
    print(name)
    print("Path:", path)
    print("Nodes expanded:", nodes)
    print("Time:", elapsed)
    print()
begin = time.perf_counter()
path, cost, nodes = ucs(start, goal)
elapsed = time.perf_counter() - begin
print("UCS")
print("Path:", path)
print("Total cost:", cost)
print("Nodes expanded:", nodes)
print("Time:", elapsed)
```

Expected Output

The BFS and DFS paths depend on the order in which neighbours are explored.

For this example:

- BFS finds a route with the fewest edges.
- DFS finds a route according to its exploration order.
- UCS finds the minimum-cost route.

The minimum-cost route is:

$$P \rightarrow A \rightarrow C \rightarrow W$$

Total cost:

$$2 + 3 + 4 = \boxed{9}$$

This small graph is used to check whether the algorithms work correctly. Larger graphs are needed for the experimental comparison in Task 9.

TASK 9 – Record Experimental Performance

The algorithms must be tested using hospital networks containing approximately:

- 1,000 locations
- 10,000 locations
- 100,000 locations
- 1,000,000 locations

Performance Measurements

The following measurements should be recorded for each algorithm:

Measurement	Meaning
Nodes generated	Total locations discovered
Nodes expanded	Locations processed by the algorithm
Maximum frontier size	Maximum number of pending locations
Solution depth	Number of movements in the selected path
Path cost	Total travel cost
Execution time	Time taken to find the route

Experimental Result Table

Use this table to enter your actual Python results.

States	BFS time	DFS time	UCS time
1,000	To measure	To measure	To measure
10,000	To measure	To measure	To measure
100,000	To measure	To measure	To measure
1,000,000	To measure	To measure	To measure

Important: These are experimental measurements, so do not enter imaginary execution times. Run the program on the generated graphs and record the actual results.

Testing Conditions

1. Use the same graph for all three algorithms.
2. Keep the starting point and goal the same.
3. Use a fixed random seed when generating graphs.
4. Repeat each experiment at least five times.
5. Calculate the average execution time.
6. Test reachable goals, unreachable goals and cases where start equals goal.

TASK 10 – Compare Theory with Experimental Behaviour

The theoretical complexity explains how algorithms should behave as the number of locations increases. Experimental results show their actual performance when the Python programs are executed.

Comparison

BFS: It explores locations level by level. It may use more memory because many locations can remain in the queue.

DFS: It explores one path at a time. It may use less frontier memory, but it can explore an unnecessary path before reaching the destination.

UCS: It selects the lowest-cost route first. It requires additional time for priority queue operations, but it provides an optimal route when all costs are non-negative.

Theory vs Experiment

Algorithm	Theoretical behaviour	Expected observation
BFS	O(V+E)	Time increases with graph size
DFS	O(V+E)	Depends on exploration order
UCS	O((V+E)\log V)	Extra time for priority queue operations

Reasons for Differences

Experimental results may differ slightly from theoretical calculations because of:

- Graph structure and number of connections.
- Repeated or duplicate locations.
- Python execution overhead.
- Priority queue operations.
- Computer memory and processor speed.

Conclusion

Theoretical analysis gives an idea of algorithm efficiency, while experimental testing helps understand actual performance. Both are necessary to select a suitable algorithm for a growing hospital network.

TASK 11 – Recommend and Defend the Final Design

Recommended Algorithm: Uniform Cost Search (UCS)

After comparing BFS, DFS and UCS, I recommend Uniform Cost Search for the hospital medicine-delivery robot.

Reasons for Selection

1. Minimum travel cost: UCS finds the route with the lowest total travel cost.
2. Different route costs: It handles routes where corridors have different travel times.
3. Accuracy: It finds an optimal solution when all movement costs are non-negative.
4. Large networks: It can work with large hospital graphs using adjacency lists and priority queues.

Comparison with Other Algorithms

Algorithm	Advantage	Limitation
BFS	Finds shortest path by number of edges	Does not consider different travel costs
DFS	Simple and uses a recursive search	Does not guarantee the best route
UCS	Finds minimum-cost route	Requires priority queue operations

Efficiency

UCS has the graph-based time complexity:

$$O\big((V+E)\log V\big)$$

Its search memory complexity is:

$$O(V)$$

An adjacency list requires additional preprocessing space of O(V+E).

Final Recommendation

UCS is the most suitable choice because delivering medicines at minimum travel cost is more important than simply reducing the number of movements. Although it requires more processing than BFS and DFS, it provides a more useful route for hospital operations.

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge

A. Impact Analysis

In the original problem, the robot knows its exact starting location. However, after a sensor failure or power restart, it may only know that it is at one of \(k\) possible junctions.

This changes the problem because:

- The starting location is no longer exactly known.
- The robot must consider multiple possible positions.
- Its decisions must depend on available sensor information.
- It must still find the pharmacy docking station.

Therefore, the robot's knowledge becomes part of the search state.

B. Evaluation of the Original Design

The original BFS, DFS and UCS algorithms assume that the robot knows its starting position.

After the sensor failure, a single location is no longer enough to represent the state.

Instead, the state becomes a belief state, which is a collection of possible robot locations.

For example:

$$B = \{A, B, C, D\}$$

This means the robot may be at any one of these four locations.

If there are k possible locations, the number of possible belief states can theoretically reach:

$$2^k$$

Therefore, directly applying the original search method to all possible belief states may become expensive.

C. Adaptation or Redesign

I would modify the original design by introducing a belief-state search with local sensing.

Working Steps

1. The robot stores all possible current locations.
2. It observes nearby surroundings using available sensors.
3. It removes locations that do not match the observations.
4. It selects a suitable movement based on the remaining possible locations.
5. It updates its belief state after every movement.
6. It continues until it reaches the pharmacy docking station.
7.

Effect on Complexity

The original search works on individual locations. The modified approach works on sets of possible locations.

- Original search: up to O(V+E) for BFS or DFS.
- Belief-state search: potentially up to 2^V belief states in the general case.

However, if only \(k\) initial locations are possible, the robot can restrict its belief states to those reachable from the initial set. This reduces unnecessary exploration in practice.

D. Evidence-Based Defence

To check whether the modified approach is useful, test it with different numbers of possible starting locations.

Possible locations (k)	Expected behaviour
2	Easy to identify the actual position
4	Requires more observations
8	More possible combinations to check

With Local Sensor

- The robot observes nearby surroundings.
- Incorrect possible locations are removed.
- The belief state becomes smaller.
- The robot can make more accurate decisions.

Without Local Sensor

- The robot keeps more possible locations.
- It may need to explore additional paths.
- The search becomes more complicated.

Final Conclusion

I would modify the original design rather than completely replace it. By combining UCS with belief-state tracking and local sensing, the robot can handle sensor failures and still find the pharmacy. This makes the system more reliable for real hospital environments.

Final Overall Conclusion:

The assignment compares BFS, DFS and UCS for a hospital medicine-delivery robot. BFS is useful for finding paths with fewer movements, DFS is useful for exploring locations, and UCS is suitable for finding minimum-cost routes.

After considering the hospital's growing size and the possibility of sensor failure, UCS combined with local sensing and belief-state tracking is the recommended design.